Skill – Week11

Session 21:

1. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

```
  GNU nano 8.7.1                                jobs.c *
#include "jobs.h"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <errno.h>

static Job jobs[MAX_JOBS];
static int job_count = 0;
static pid_t shell_pgid;
static int terminal_fd = STDIN_FILENO;

/* Initialize shell signal actions */
void init_shell(void)
{
    shell_pgid = getpgrp();

    signal(SIGINT, SIG_IGN);
    signal(SIGQUIT, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTTOU, SIG_IGN);

    printf("Shell PID: %d\n", getpid());
    printf("Shell PGID: %d\n", shell_pgid);
}

/* Add a new job */
static void add_job(pid_t pid)
{
    if (job_count < MAX_JOBS)
    {
        jobs[job_count].pid = pid;
        jobs[job_count].pgid = pid;
        jobs[job_count].status = RUNNING;
        job_count++;
```

```
  GNU nano 8.7.1                                main.c *
#include <stdio.h>
#include "jobs.h"
int main(void)
{
    int choice;
    init_shell();
    while (1)
    {
        printf("\n===== MINI JOB CONTROL SHELL =====\n");
        printf("1. Create Foreground Job\n");
        printf("2. Show Job Table\n");
        printf("3. Resume Stopped Job\n");
        printf("4. Terminate Job\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input.\n");
            break;
        }
        switch (choice)
        {
            case 1:
                create_job();
                break;
            case 2:
                show_jobs();
               break;
            case 3:
                resume_job();
                break;
            case 4:
                terminate_job();
               break;
            case 5:
                cleanup_jobs();
                printf("Shell exited safely.\n");
```

```
manikanta@Hemu:~/job_control_project$ ./job_control
Shell PID: 581
Shell PGID: 581

===== MINI JOB CONTROL SHELL =====
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Terminate Job
5. Exit
Enter choice: 1
Foreground job started. PID: 582
Press Ctrl+C to terminate or Ctrl+Z to suspend.
Child 582 running: 1
Child 582 running: 2
Child 582 running: 3
Child 582 running: 4
Child 582 running: 5
^ZJob suspended.

===== MINI JOB CONTROL SHELL =====
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Terminate Job
5. Exit
Enter choice: 1
Foreground job started. PID: 585
Press Ctrl+C to terminate or Ctrl+Z to suspend.
Child 585 running: 1
Child 585 running: 2
Child 585 running: 3
^ZJob suspended.
```

```
Enter choice: 2

--- JOB TABLE ---
Job 1 | PID: 582 | PGID: 582 | Stopped
Job 2 | PID: 585 | PGID: 585 | Stopped

===== MINI JOB CONTROL SHELL =====
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Terminate Job
5. Exit
```

Report: The modules were integrated using a shared header file and separate source files. End-to-end testing verified process creation, signal handling, terminal control, job-state updates, suspension, resumption, error handling, and cleanup.

2. To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```
  GNU nano 8.7.1                          errors.c *
#include "errors.h"

#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <time.h>
#include <stdlib.h>

/* Log an application error */
void log_error(const char *message)
{
    FILE *file = fopen("error.log", "a");

    if (file == NULL)
    {
        perror("Unable to open error.log");
        return;
    }

    time_t now = time(NULL);
    char *timestamp = ctime(&now);

    if (timestamp != NULL)
    {
        timestamp[strcspn(timestamp, "\n")] = '\0';

        fprintf(file, "[%s] ERROR: %s\n",
                timestamp, message);
    }
    else
    {
        fprintf(file, "ERROR: %s\n", message);
    }

    fclose(file);

    fprintf(stderr, "Error: %s\n", message);
```

```c
#include <stdio.h>
#include "jobs.h"
#include "errors.h"

int main(void)
{
    int choice;
    int result;

    init_shell();

    while (1)
    {
        printf("\n===== MINI JOB CONTROL SHELL =====\n");
        printf("1. Create Foreground Job\n");
        printf("2. Show Job Table\n");
        printf("3. Resume Stopped Job\n");
        printf("4. Terminate Job\n");
        printf("5. Test Invalid Input\n");
        printf("6. Exit\n");

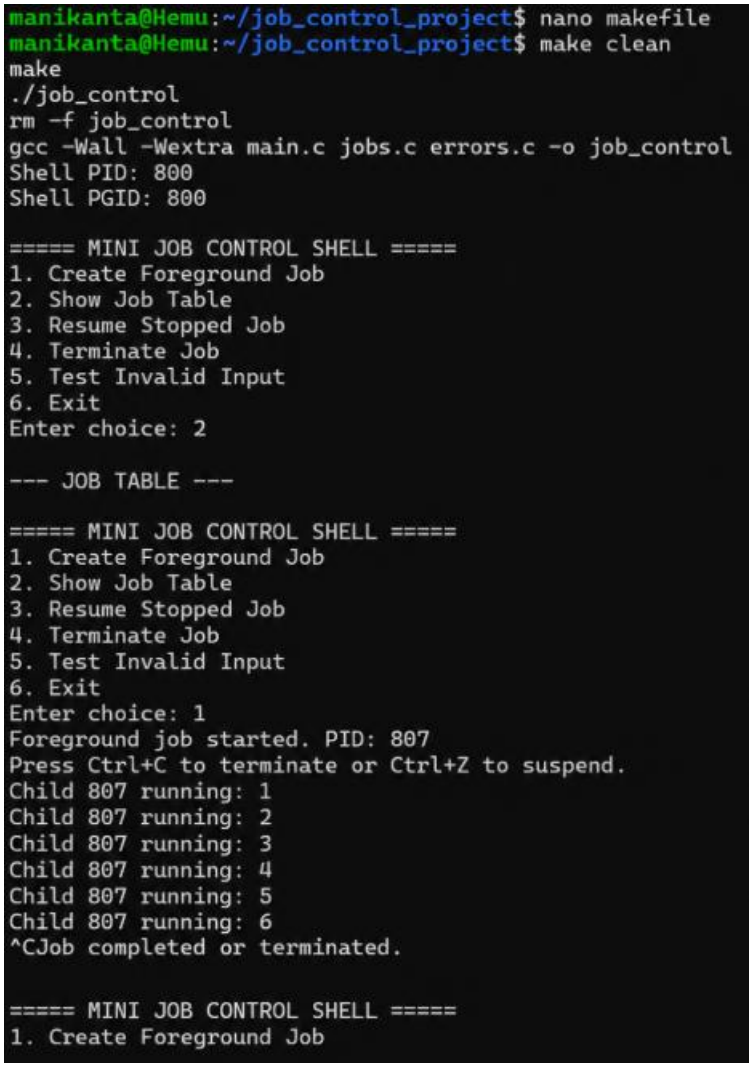
        result = read_choice(&choice);

        if (result == 0)
        {
            printf("\nInput closed. Cleaning up.\n");
            break;
        }

        if (result == -1)
        {
            printf("Please try again.\n");
            continue;
        }

        switch (choice)
        {
```

Output:



Report: The mini shell was tested using gcc -Wall -Wextra, ./job_control, and cat error.log.

Session 22:

1. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```
  GNU nano 8.7.1                                memory1.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = malloc(5 * sizeof(int));

    if (ptr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        ptr[i] = i + 1;
        printf("%d ", ptr[i]);
    }

    printf("\n");

    return 0;
}
```

```
manikanta@Hemu:~/OS_Lab$ nano memory1.c
manikanta@Hemu:~/OS_Lab$ gcc -g -Wall -Wextra memory1.c -o memory1
manikanta@Hemu:~/OS_Lab$ ./memory1
1 2 3 4 5
manikanta@Hemu:~/OS_Lab$ valgrind --leak-check=full --show-leak-kinds=all \
./memory1
==1090== Memcheck, a memory error detector
==1090== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1090== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1090== Command: ./memory1
1 2 3 4 5
==1090==
==1090== HEAP SUMMARY:
==1090==     in use at exit: 20 bytes in 1 blocks
==1090==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==1090==
==1090== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==1090==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==1090==    by 0x40011BE: main (memory1.c:5)
==1090==
==1090== LEAK SUMMARY:
==1090==    definitely lost: 20 bytes in 1 blocks
==1090==    indirectly lost: 0 bytes in 0 blocks
==1090==      possibly lost: 0 bytes in 0 blocks
==1090==    still reachable: 0 bytes in 0 blocks
==1090==         suppressed: 0 bytes in 0 blocks
==1090==
==1090== For lists of detected and suppressed errors, rerun with: -s
==1090== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Report: The C program was compiled using gcc -g -Wall -Wextra and tested using ./memory. Valgrind detected the memory leak caused by missing free().

2. To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```
  GNU nano 8.7.1                          calculator.c *
#include <stdio.h>

int main() {
    int a, b;

    scanf("%d %d", &a, &b);

    printf("%d\n", a + b);

    return 0;
}
```

```
manikanta@Hemu:~/OS_Lab$ nano calculator.c
manikanta@Hemu:~/OS_Lab$ gcc -Wall -Wextra calculator.c -o calculator
manikanta@Hemu:~/OS_Lab$ nano test.sh
manikanta@Hemu:~/OS_Lab$ chmod +x test.sh
manikanta@Hemu:~/OS_Lab$ ./test.sh
TC1: Expected=8 Actual=8 PASS
TC2: Expected=30 Actual=30 PASS
TC3: Expected=0 Actual=0 PASS
TC4: Expected=0 Actual=0 PASS
Test report generated: report.csv
manikanta@Hemu:~/OS_Lab$ cat report.csv
Test Case,Expected,Actual,Result
TC1,8,8,PASS
TC2,30,30,PASS
TC3,0,0,PASS
TC4,0,0,PASS
```

Report: Test cases were designed and executed automatically using a Bash script. The program was compiled using gcc, and its outputs were compared with expected results. All four test cases passed, and the results were saved in report.csv using cat report.csv for verification.